System Verification and Validation Plan for Software Engineering

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

October 27, 2025

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

List of Tables

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

Refer to Section 4 in the SRS document for a table of many more symbols, abbereviations, acronyms, and definitions.

This document aims to outline the verification and validation that the Physiotherapy Companion app will go through during development. Starting with what the software is, what it is meant to achieve, and some relevant information before explaining in detail the standards and metrics that will be used to evaluate the software.

2 General Information

2.1 Summary

The software focused on in this document is the Physiotherapy Companion application, built for smart devices with camera access to assess a user, expected to be a patient undergoing physiotherapy, on their form during assigned physiotherapy exercises.

The application will provide support and guidance for the exercise and make real-time suggestions to improve the user's form during a video-feed. It will also provide overviews of previous exercises to allow users to see progress and consistency, assisting in forming healthy habits towards recovery and reinforce confidence.

The application will also support a more administrative side for physiotherapists, allowing them to view exercise overviews for patients assigned to them, as well as leave comments on overviews to ensure consistent and timely usage of the application for healthy exercising.

2.2 Objectives

The objectives of the testing expanded upon within this document focus on general system usability, the security of user profiles and video-feed data, and the verification of stakeholder requirements.

Usage of the system should be easily explainable for first-time users with simple dialogue popups and limited visible guiding from one function to the next. Additionally, for the video-feed portion of the application's usage, the user should be reasonably guided through the exercise and process to set them up for successful exercise execution and form recommendation acceptance.

User profile and video-feed data should be securely stored, whether locally or within a remote database accessed by the system. User-data has become an increasingly sensitive topic within the software industry of late

and everyone wants to know how data is used and dispersed. In this way, users should be aware of the usage of their data and the system should ensure unauthorized access outside of the communicated plan is never possible.

Meeting stakeholder requirements, whether they be data restrictive or process flow in nature, is what will validate the core features of the system. If the users the system is designed for don't find the usage expected from the system, then ultimately the system has failed to perform its primary goal. Ensuring stakeholder requirements and feedback are incorporated into the system as it is developed, both initially and onward, will ensure the system is both wanted and usable to the intended audience.

Some out-of-scope objectives include performance and user-interface quality. In this sense, quality means the overall refinement and how finalized and polished the interface looks; the implementation should not inhibit usability, but every button need not be graphically developed individually.

For performance, the system should respond in a timely manner that doesn't frustrate a user's ability to perform the main activities of the system, but using the most optimal algorithm and fully optimizing the system are not direct objectives of the system at this time. These things could both be expanded into and developed, but for now are considered out of scope.

2.3 Extras

The extras set out for the system consist of a Design Thinking Document and a User Instructional Video.

The Design Thinking Document aims to explain the process behind, and work done towards, the final design of the system. This document explains the overall flow of processing through the system but also the interfacing elements, external libraries utilized, and more. The document flow is iterative, as is the design process, and aims to communicate the entire design process flow of the system.

The User Instructional Video aims to communicate the ideal usage of the system to the primary stakeholders. The video will demonstrate the usage of the analytics display, the video-feed submission function along with form recommendations, as well as an explanation of the overviews provided from each completed exercise. All of these will be instructed from the perspective of a patient user within the system. For the physiotherapist view, the patient list and comment features will be focussed on along with the analytical display function previously focussed on.

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

This section will focus on the methods to verify and validate artifacts of the project. The methods for each artifact or facet of the system will be listed below, and additional information including possible questionnaire questions can be found in the appendix at the end of this document. The section will start primarily with verification and move to validation at the end.

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

To verify the SRS, several approaches will be used across different sections of the document. In general, the document will receive verification through feedback from 4G06 teaching assistants, peer-review feedback from another design team, and a check in with a project advisor to ensure the document represents the wants for the project. This check-in will be a self review by the advisor followed by a scheduled meeting to present any requirements felt could possibly change. Based on feedback from the advisor, issues will be created and handled.

To verify the functional requirements within the SRS, the above will all be used in addition to software checks for functionality and a requirements

checklist to be updated upon checking functional requirements within the implementation. For verification of non-functional requirements within the SRS, again including all the general checks, there will be a questionnaire of the software's usage and another checklist with a less binary scaling system for the NFRs.

3.3 Design Verification

To verify the design we'll rely on feedback from 4G06 teaching assistants, from peer-review feedback provided by colleagues on another design team, as well as advisor and stakeholder questioning. The feedback from teaching assistants and peer-review feedback will be created as issues within our repository and handled accordingly. For advisor and stakeholder questioning, there will be software handouts along with a questionnaire which is expanded on more at the end of this section.

3.4 Verification and Validation Plan Verification

This document will also be reviewed by the teaching assistants and provided peer-review by another design team, again creating issues within the GitHub repository to be dealt with accordingly.

In addition, the test suite derived from this document will be verified via code coverage and mutation testing on the software itself. For validation, the final outcome for each test will be discussed in the previously mentioned meeting with the project advisor to ensure the system is doing what it should upon each test case.

3.5 Implementation Verification

For testing the implemented system, there will be a software testing suite developed based on the tests listed in sections 4 and 5 of this document. Additional testing by the team includes code walk-throughs of critical sections both during development and upon reaching milestones within the development cycle. Throughout development, there will also be static analysis on each pull request to GitHub, ensuring naming conventions and best practice are followed throughout the project.

3.6 Automated Testing and Verification Tools

Automated testing includes the testing suite previously mentioned in this section; derived from the tests found in this document which are based upon the requirements found within the SRS document of this project. These tests will be verified themselves with mutation testing and code coverage assessments. A testing suite such as pytest or Python Extension will be utilized to implement the tests mentioned in further sections of this document. These suits contain their own internal tools for checking code coverage, and the GitHub will be using continuous integration to ensure base cases continue to pass through all pull requests as well as general compilation checks.

During development, pylint will ensure the coding standard is adhered to constantly and also be working with the continuous development within the GitHub to ensure no slip-ups on merging.

3.7 Software Validation

For software validation, to ensure the proper system was designed to solve the problems addressed at the beginning of this project and outlined within the problem statement document, there will be system walkthroughs and interviews with stakeholders and the project advisor. Within these interviews we will assess time to use the systems functions, understanding of the system and how it processes data, and general acceptance of the system. The interview will end with a few questions outlined in the appendix of this document.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

Output:

How test will be performed:

4.2.2 Area of Testing2

...

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all of the modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

6 Appendix

Additional project information to expand on the previous sections. Mostly example or sample information at this time.

6.1 Symbolic Parameters

Example script layout for system testing:
/*
Create user account testing
New account creation (user_acc_create)
Invalid format of field inputs (invalid_formatCreate)
Invalid sign up credentials (cannot_verify_creds)
Presence of an existing account (acc_exists)

PT focused testing
PT successfully sends invite to patient (PT_sends_invite)
PT adds/modified patient exercise (PT_adjusts_exercise)
PT deletes patient account (PT_deletes_patient_acc)

Patient focused testing
System_exercise_overview
Video_store
Video_retrieval
System_analysis_feedback
Post_ex_patient_feedback
Incorrect_form_adjustment

NFR testing
Ex_int_speed
Fail_recovery
Interrupted_vid
/

6.2 Usability Survey Questions?

Advisor SRS Questions:

1. Are there any sections of the requirements which are found to be confusing or hard to measure?
2. Are there any sections of the system you feel the requirements do not expand on?
3. Are there any sections of the system you feel the requirements expand on that is out of scope?
4. Do you feel the requirements are extensive enough?
5. Do you feel the requirements are tracked well?

Design Questionnaire Questions:
"Strongly Disagree" to "Strongly Agree" style questioning.

1. The system navigation is intuitive.
2. The system's functions are all easily accessible.
3. The system helped set up your exercises for success.
4. The system enables you to complete your exercises with better guidance.
5. The system enables you to complete your exercises with better confidence.
6. The feedback returned during your exercise was helpful.
7. The overview data from your exercise was helpful.
8. You would you use this software for your exercises again.

Could have a final open-ended question based on user pain points with the system. Ie. what didn't work well for individuals.

Software Validation Questionnaire Questions:
"Strongly Disagree" to "Strongly Agree" style questioning.

1. This software assisted you in completing your exercises.
2. This software was easy to use.

3. This software was easy to navigate.
4. The feedback from the system assisted you in fixing your form.
5. The feedback from the system generated greater confidence in your form.
6. The feedback from the system was meaningful and helpful.
7. The data generated from your exercises was helpful.
8. The system responded how you expected it to during usage.
9. The system clearly described what was going on.
10. You felt adequately trained to use the system from the system dialogues.
11. You felt safe while using the system.

Ui_testing Survey:

1. How would you rate your overall experience with the application (1 - poor, 5 - excellent)?
2. How would you rate the visual appeal of the application (1 - poor, 5 - excellent)?
3. How easy was it to navigate through the application? (1 - difficult, 5 - easy)
    (a) Were the tutorials helpful? (1 - not helpful, 5 - very helpful)
    (b) How logical was the arrangement of the different components within the system (1 - very illogical, 5 - very logical)?
4. Which areas could use some improvement?
    (a) Navigation
    (b) Menu structure
    (c) Colour/contrast scheme
    (d) Error messages

(e) Other (With open text entry)

Patient Post-Exercise Feedback Form:

1. How did you feel before the exercise? (1 - good, 5-bad)
   (a) Did you have tightness? Yes/no
   (b) Did you have any pain? Yes/no.
      i. How bad was your pain? (1 - no pain, 10 - unbearable pain)
      ii. What was the perceived level of pain/difficulty? (1 - very easy, 10 - very difficult)
      iii. Where was your pain located? (Open text entry)
   (c) Have you noticed any recent changes? (Open text entry)
2. If you had any pain DURING the exercise, how bad was it? (1 - no pain, 10 - unbearable pain)
3. How did you feel after the exercise? (1 - good, 5-bad)
   (a) Did you have tightness? Yes/no
   (b) Did you have any pain? Yes/no.
      i. How bad was your pain? (1 - no pain, 10 - unbearable pain)
      ii. What was the perceived level of pain/difficulty? (1 - very easy, 10 - very difficult)
      iii. Where was your pain located? (Open text entry)
   (c) Have you noticed any recent changes? (Open text entry)
4. How challenging was the exercise today? (1 - easy, 5 - very hard)
   (a) Do you have any feedback for your physiotherapist? (Open text entry)
5. In your opinion, is this treatment plan helping your health? Yes/no. Choose the option that describes how the plan is helping (Can choose multiple).
   (a) My movement
   (b) My flexibility

(c) My pain

(d) My mood

6. In your opinion, how difficult is the current prescribed exercise plan? (1 - easy, 5 - very difficult)

   (a) Should your physiotherapist change the difficulty of your plan? (1 - no change, 5 - Large increase in difficulty)

Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?